# Leios Voting Cryptographic Review

ARC Applied Cryptography Team

April 24, 2026

## Abstract

This document aims at exploring different committee selection algorithms for Leios protocol certificate generation. In passing, we will add notes and remarks on the security of existing designs, such as the use of VRFs, and tentative design choices made for the Leios prototype.

## 1 Introduction

Leios design aims at increasing Cardano's throughput by decoupling transactions from the main chain. Transactions are now additionally collected and endorsed in endorsement blocks (EBs), and validated through certificates, which are referred on the main chain through ranking blocks (RBs).

To ensure data availability and correctness, the EBs are referred to the permanent ledgers in the RBs if and only if the network has validated them. For performance, a smaller set of nodes, a committee, is selected and asked to vote on the validity of these blocks. The committee being much smaller than the network, fewer votes will be disseminated that way. Cryptographic primitives will furthermore be used to aggregate the votes into a single succinct certificate for faster verification and smaller size.

The committee validation is chosen through multiple considerations. First, we want legitimate EBs to be endorsed, that is certificates to be generated, with good probability. Reciprocally, we do not want adversaries to prevent the certification of a valid EB. Additionally, we want erroneous certificates, valid certificates on incorrect EBs, not to be generated with strong probabilities. To do so, we require the committee size and the threshold for vote aggregation to be large enough so that an adversary with a set stake cannot perform these attacks with significative probabilities (certificate prevention, malicious generation).

Analysis of committee selection done through IID selection (SPOs chosen relative to their stake with a binomial distribution) and diffusion showed that a threshold of 75% of the *active* stake and a committee size of 600 members are satisfiable. These numbers were chosen such that the committee comprises enough of the total stake, an adversary with under 50% of the total stake cannot perform the attacks, and that a sizable portion of honest users, here 25% have

seen the EB before it is certified on-chain. We can however note that, without networking consideration, a threshold of 50.1% would technically be enough and the first technical report recommended only a threshold of 60%.

Now that the committee size and vote aggregation threshold are set, we can look at the committee selection protocol itself. Some probabilistic algorithms called lotteries, such as IID (c.f. Section 2.4). mentioned before or sortition [7] (c.f. Section 2.5), allow for private elections. This means that the selection of the committee member is done locally and privately. The advantage of doing this is that this selection cannot be foreseen and so, helps with attacks on the committee members such as DoS. It, however, also comes with caveats, notably, the committee member must also include in their vote a proof that they won the committee selection. This in effect double the vote size and verification. The Fait Accompli paper (FA [5], Section 4.1) in 2023 shows how the bias of the stake distribution can be used to our advantage. It presents several designs showing how to determine whether a SPO is wealthy enough to always (with overwhelming probabilities) be selected as committee member and so remove the need for these to generate an eligibility proof. By selecting these members *deterministically*, FA furthermore outperforms IID and sortition in security. Committees can be chosen of smaller size, to a certain extent, without the risk of over-representing some parties. (Note that the threshold was increased for diffusion which is independent of the committee selection security however.) We shall talk about security of FA committee size in Section 4.1.

# 2 Preliminaries

We first present here the notations and cryptographic primitives that we will use in this document.

## 2.1 Notations

We define by $\{x_i\}_n$ an unordered set of $n$ elements, and by $[x_i]_m$ an ordered array, or vector, of $m$ elements. To access the $j^{th}$ element of a vector vec, we use the notation vec$[j-1]$, the first element of either data structure having index 0. We sometimes use $[a;n]$ to denote an array of $n$ elements, all initialized with value $a$. $[n] = [0,n)$ denotes the sequence $0,...,n-1$.

Let $\mathcal{G}$ be a probabilistic polynomial time algorithm that on input $1^\kappa$ returns $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$, where $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ are groups of prime order $p$; where $g_1$ and $g_2$ generate $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively and where $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable, non-degenerate bilinear map.

Unless stated otherwise, $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2) \xleftarrow{\$} \mathcal{G}(1^\kappa)$ will denote the safe subgroup of BLS12-381 curve and $\mathbb{F}_p$, its associated scalar field.

We denote by $h \leftarrow \mathsf{Hash}_{\mathbb{X}}(\mathrm{m})$ the hash function $\mathsf{H}$ taking as input m and returning the digest $h \in \mathbb{X}$. We may instantiate $\mathsf{H}$ directly, e.g. SHA256, or define it over a group, e.g. $g \leftarrow \mathsf{Hash}_{\mathbb{G}_1}(\mathrm{m})$.

## 2.2 BLS multi-signatures

We succinctly describe here the functionality of BLS signatures [1].

**Key generation.** Algorithm 1 describes the generation of BLS keys, used for signing votes.

**Algorithm 1** BLS.KeyGen

**Output:** $\mathrm{sk}_{bls},\ \mathrm{vk}_{bls}$

1: $\mathrm{sk}_{bls} \xleftarrow{\$} \mathbb{F}_p;\ \mathrm{vk}_{bls} \leftarrow g_2^{\mathrm{sk}_{bls}}$
2: **return** $(\mathrm{sk}_{bls},\ \mathrm{vk}_{bls})$

**Remark.** We made the arbitrary choice to define verifying keys in $\mathbb{G}_2$, and signatures in $\mathbb{G}_1$, but these can be swapped without loss of generality.

**Signature generation and verification.** Algorithm 2 describes the generation of BLS signatures and Algorithm 3 their verification.

**Algorithm 2** BLS.Sign

**Input:** $\mathrm{sk}_{bls},\ \mathrm{m}$
**Output:** $\sigma$

1: **return** $\mathsf{Hash}_{\mathbb{G}_1}(\mathrm{m})^{\mathrm{sk}_{bls}}$

**Algorithm 3** BLS.Verify

**Input:** $\mathrm{vk}_{bls},\ \mathrm{m},\ \sigma$
**Output:** $\top/\bot$

1: **return** $e(\sigma, g_2) \stackrel{?}{=} e(\mathsf{Hash}_{\mathbb{G}_1}(\mathrm{m}), \mathrm{vk}_{bls})$

**Signature Aggregation.** Algorithm 4 defines a key batching algorithm and Algorithm 5 the corresponding signature batching algorithm such that the aggregated signature verification succeeds, i.e. $\mathsf{BLS.Verify}(\mathrm{m}, \sigma_{agg}, \mathrm{vk}_{agg}) = 1$, with overwhelming probability if and only if each individual signature is valid.

**Algorithm 4** BLS.BatchKey

**Input:** $\{\mathrm{vk}_i\}_n$
**Output:** $\mathrm{vk}_{agg}$

1: $\alpha \leftarrow \mathsf{Hash}_{\mathbb{F}_p}(\{\mathrm{vk}_i\}_n)$
2: **return** $\prod_i \mathrm{vk}_i^{\alpha^i}$

**Algorithm 5** BLS.BatchSig

**Input:** $\{\mathrm{vk}_i\}_n, \{\sigma_i\}_n$
**Output:** $\sigma_{agg}$

1: $\alpha \leftarrow \mathsf{Hash}_{\mathbb{F}_p}(\{\mathrm{vk}_i\}_n)$
2: **return** $\prod_i \sigma_i^{\alpha^i}$

## 2.3 KES signatures

For the purpose of this document, we describe a KES signature scheme as a tuple $(\mathsf{Sign}, \mathsf{Verify})$ where,

- $\sigma \leftarrow \mathsf{KES.Sign}(\mathrm{sk}_{kes}, \mathrm{m})$ produces a signature $\sigma$ over message m given a private key $\mathrm{sk}_{kes}$, and
- $\top/\bot \leftarrow \mathsf{KES.Verify}(\mathrm{vk}_{kes}, \mathrm{m}, \sigma)$ returns $\top$ if $\sigma$ is a valid KES signature over $\mathrm{vk}_{kes}$ and m, or $\bot$ otherwise.

Properties of interest include that KES signatures can be rotated but cannot be batched. For more details on KES signatures, one can read [8].

## 2.4 IID selection IID

The IID selection is named after the fact that we consider a lottery of $n$ *independent and identically distributed* seats to assign to $m$ weighted parties, with corresponding weight $\{\mathrm{s}_i\}_m$.

To assign them, we can use the multinomial distribution to compute the total number of seats, out of $m$, for each of the $n$ parties:

$$(X_1, \ldots, X_n) \sim \mathsf{Multinomial}(m; p_1, \ldots, p_n)$$

where $p_i = \mathrm{s}_i / \sum_j \mathrm{s}_j$. In practice however, as shown in Algorithm 6, we may for each $m$ seat run a fair coin toss on $[0, \sum_i \mathrm{s}_i)$ and assign the seat to party $k$ if the coin falls in interval $\left[\sum_{i=0}^{k} \mathrm{s}_i, \sum_{i=0}^{k+1} \mathrm{s}_i\right)$.

**Algorithm 6** IID.Eval

**Input:**
- $n$: Committee size
- $\{\mathrm{s}_i\}_m$: Weights
- $r$: random nonce

**Output:**
- $\{w_i\}_m$: Number of winning tickets for all $m$ parties

```
s_tot ← Σ_i s_i
weights ← [0; m] // An array of m slots, all initialized to 0
for j ∈ [0, n] do
    i ← k such that (s_tot · Hash_N(r‖j)) / N ∈ [Σ_{i=0}^{k} s_i, Σ_{i=0}^{k+1} s_i)
    weights[i] ← weights[i] + 1
return weights
```

We can note that this algorithm is *public*. The only knowledge for computing the whole committee selection is the ordered stake distribution and the nonce. We could however turn this algorithm private, by having each party generate a verifiable random value, through a VRF or BLS signature on an election identifier for instance, used as a nonce on $n$ coin tosses with probability $\mathrm{s}_i / \sum_j \mathrm{s}_j$. For a party to prove that they have won $w$ seats, they would provide the random

value value that would be verified with their verifying key and the number of seats finally would be recomputed out of it.

## 2.5 Sortition LS

The sortition algorithm, presented in the Algorand paper [7] is a cryptographic mechanism used to randomly and securely select members of a committee.

Instead of a public check as in I.I.D., sortition is an algorithm where participants can check *locally* and *privately* whether they are part of the committee and whose eligibility can still be checked in public. To achieve this asymmetry, sortition relies on the usage of verifiable random functions (VRFs) that allow for the private verifiable generation of random numbers.

Private elections have the merit of greater security as committee members are unknown until their votes are broadcast. As such, they cannot be specifically targeted by DoS or other attacks.

In practice, we may rely on BLS signatures instead of VRF as they have sufficient randomness properties and to reduce the number of keys and algorithm to handle and maintain.

**Sortition evaluation.** As shown in Algorithm 7, evaluating with local sortition whether one is part of a committee consists in generating a *verifiable* random number *deterministically* from one's secret key and the election identifier and comparing it with the probability distribution of winning a number of lottery tickets given one's stake.

In practice, we compare the normalized hash digest of the BLS signature to increasing intervals of the cumulative distribution function of the Binomial distribution. The order of the interval the normalized digest is included in will correspond to the number of winning tickets.

**Algorithm 7** LS.Eval

**Input:**

- $n$: Committee size
- $eid$: Election ID
- s: Caller's stake share
- sk: Caller's signing key.

**Output:**

- $w$: Number of winning tickets for owner of sk
- $\sigma_{eid}$: Proof of eligibility

1: $\sigma_{\text{eid}} \leftarrow \mathsf{BLS.Sign}(eid, \text{sk})$
2: $p \leftarrow \mathsf{SHA256}(\sigma_{\text{eid}})\ /\ 2^{256}$ // Normalized to $[0, 1]$
3: $w \leftarrow i\ s.t.\ p \in \left[\sum_{k=0}^{i} \mathsf{B}(k,\ \text{s},\ n), \sum_{k=0}^{i+1} \mathsf{B}(k,\ \text{s},\ n)\right)$
4: **return** $(w,\ \sigma_{\text{eid}})$

**Remark.** A prototype of sortition was implemented by Brian Bush, c.f. [2, src/main.rs#304]. In this prototype, the normalization step is tweaked to use 256 bit long integers and so the hash digest is divided by $2^{256} - 1$ instead which leads to a negligible loss in soundness.

**Sortition verification.** As shown in Algorithm 8, verifying the eligibility of a person in a committee follows sortition's evaluation, except that the proof of elibigility is provided and checked – that is, the BLS signature is verified.

**Algorithm 8** LS.Verify

**Input:**

- $n$: Committee size
- $eid$: Election ID
- s: Caller's stake share
- vk: Caller's signing key.
- $\sigma_{\text{eid}}$ proof of eligibility

**Output:**

- $w$: Number of winning tickets for owner of vk
- $\top \vee \bot$: boolean whether the person is eligible or not

1: $b \leftarrow \mathsf{BLS.Verify}(eid, \sigma_{\text{eid}}, \text{vk})$
2: $p \leftarrow \mathsf{SHA256}(\sigma_{\text{eid}})\ /\ 2^{256}$ // Normalized to $[0, 1]$
3: $w \leftarrow i\ s.t.\ p \in \left[\sum_{k=0}^{i} \mathsf{B}(k,\ \text{s},\ n), \sum_{k=0}^{i+1} \mathsf{B}(k,\ \text{s},\ n)\right)$
4: **return** $(w,\ b \wedge w \neq 0)$

# 3 Candidate Protocols

## 3.1 High-level overview

Figure 1 sketches the stages for voting in Leios with regular committee selection. All options considered next follow it – the difference being only the way in which individual SPOs are considered eligible to vote or not.

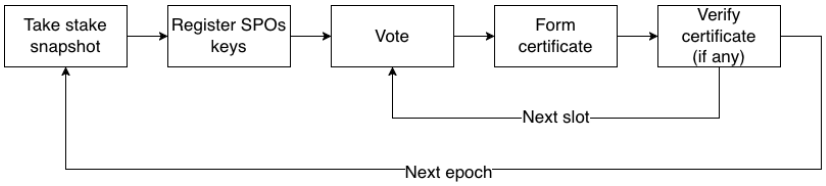


Figure 1: High-level process for voting in Leios.

## 3.2 Key management

For efficient aggregation of votes into certificates in Leios, the current proposals rely on BLS signatures. The variants that rely on locally computable eligibility checks use local sortition LS. Both for signing votes and eligibility checks, SPOs need to generate and register the needed keys. Since this is a common process to all voting variants that we cover next, we introduce next the main algorithms for handling key registration.

After introducing these common aspects, we move to describing the different ways in which each proposal picks the voting committee.

**Key registration.** After generating the BLS keys, they need to be registered. For this, all proposed variants rely on KES signatures, which are already integrated within Cardano, and each SPO is required to maintain a KES key pair. Algorithm 9 defines how to create a key registration request, based on [2, `src/key.rs#164`],[3], while Algorithm 10 defines how to validate a key registration – updating the state of accepted keys.

To register a key, see Algorithm 9, we require a proof of possession of the newly generated secret key $\mathrm{sk}_{bls}$ and an unforgeable link between one's KES keys and their BLS keys. Linkage to the cold key directly is another option. We compute the first one by generating the classic "POP" signature as well as the reciprocal verifying key for robustness. As for the latter, a fresh KES signature on the BLS key is sufficient. Note: We made the arbitrary choice to add to the KES signature the epoch number $n$, to ensure that the KES key is valid at that time – this may not match the current implementation.

**Algorithm 9** KeyReg.Sign

**Input:** $\mathrm{sk}_{bls},\ \mathrm{vk}_{bls},\ \mathrm{sk}_{kes},\ \mathrm{n}$

**Output:** $pop_{bls},\ \sigma_{kes}$

1: $\sigma \leftarrow \mathsf{BLS.Sign}(\text{"}Pop\text{"} \mid\mid \mathrm{vk}_{bls},\ \mathrm{sk}_{bls})$
2: $\mathrm{vk}_{bls,\mathbb{G}_1} \leftarrow g_1^{\mathrm{sk}_{bls}}$
3: $pop \leftarrow (\sigma,\ \mathrm{vk}_{bls,\mathbb{G}_1})$
4: $\sigma_{kes} \leftarrow \mathsf{KES.Sign}(\mathrm{n} \mid\mid \mathrm{vk}_{bls}, \mathrm{sk}_{kes})$
5: **return** $(pop,\ \sigma_{kes})$

To verify and accept a registration request, as seen in Algorithm 10, we must first ensure that the KES key is known and valid for epoch $n$ with regards to the local ordered stake distribution OSD. We then check both BLS and KES signature and update our local state if all checks have passed successfully.

**Algorithm 10** KeyReg.Verify

**Input:** $\text{state}, \text{n}, \text{poolID}, \ \text{vk}_{kes}, \ \text{vk}_{bls}, \ pop, \ \sigma_{kes}$
**Output:** state

$b_{pool} \leftarrow \text{OSD}[i].\text{vk}_{kes} \stackrel{?}{=} \text{vk}^i_{kes}$
$b_{pop} \leftarrow \mathsf{BLS.Verify}(pop, \text{“}Pop\text{”} \ || \ \text{vk}_{bls}, \text{vk}_{bls})$
$b_{kes} \leftarrow \mathsf{KES.Verify}(\sigma_{kes}, \text{n} \ || \ \text{vk}_{bls}, \text{vk}_{kes})$
**if** $b_{pool} \wedge b_{pop} \wedge b_{kes} =$ **true then**
    Update state with $\text{vk}_{bls}$ as the key for pool identified by poolID
**return** state

**Remark.** Note that we did not verify that a BLS key has already been registered. This is because the probability of sampling the same key is negligible and there is no advantage for an adversary in reusing the same key.

## 3.3 Committee Selection

We present here generic committee selection interfaces that we will instantiate later with different options. We assume two kind of voters: persistent (PVs) and non-persistent ones (NPVs). The former will be able to vote for a fixed extended period of time while the other one on a case by case basis. Both may present a proof of eligibility, which may be $\perp$, together with their vote to confirm whether they were supposed to vote.

**Remark.** We assume that all parties maintain the stake distribution frozen at the end of the previous epoch – that is, the list of $(p_i, \text{vk}^i_{\text{bls}}, \text{s}_i)$ tuples, mapping parties to their verification keys and normalized stake, in non-increasing order per stake distribution.

**Setting up elections.** The committee selection setup algorithm, c.f. Algorithm 11, is used for setting the parameters of selecting a committee for a period of time.

Given an ordered, in non-increasing way, stake distribution OSD, that is $[(p_i, \text{vk}^i_{\text{bls}}, \text{s}_i)]$ pairs for non-increasing $\text{s}_i$ (and $\text{vk}^i_{\text{bls}}$ if the stakes are not unique), and a target committee size $n$ that may have been derived from a target cumulative stake, the algorithm returns the number of persistent voters $n_1$ and information about them. We can note that if $n_1 = n$, there are no non-persistent voters. Likewise, if $n_1 = 0$, there are no persistent ones.

**Algorithm 11** Committee.Setup

**Input:**

- OSD: List of $(p_i, \mathrm{vk}^i_{\mathrm{bls}}, \mathrm{s}_i)$ pairs, mapping parties to their verification keys and normalized stake, in non-increasing order per stake distribution,
- $n$: Target committee size.

**Output:**

- $n_1$: The number of persistent voters ($n - n_1$ being the number of non-persistent voters),
- $[(p_1, w_1), \ \ldots, (p_{n_1}, w_{n_1})]$: The weighted list of persistent voters.

**Determining eligibility.** The committee selection evaluation algorithm, c.f. Algorithm 12, is used for determining and proving if oneself is eligible for a particular election *eid*. This algorithm returns a boolean showing eligibility, showing the number of lottery tickets won, and a proof of eligibility.

**Algorithm 12** Committee.Eval

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \mathrm{PVs})$: Target committee size, and number and list of PVs,
- *eid*: Election ID,
- sk: Caller's private key.

**Output:**

- $b$: boolean set to $\top$ if the caller may vote, $\perp$ otherwise,
- $\pi$: proof of eligibility, may be $\perp$ if not needed.

**Remark.** Note that the weight of the vote is not included in the output. This is because the verification process is expected to recompute this value, and it is not required when determining eligibility. Furthermore, we assume that when $b = 1$, the weight is non-null, and that the verification algorithm (c.f. Algorithm 13) will produce the weight as part of its output. Doing so, there is no need to include the weight within the votes themselves.

**Verifying eligibility.** The committee selection verification algorithm, c.f. Algorithm 13, is used for checking if a party is indeed eligible for a particular election *eid*. This algorithm returns a boolean showing eligibility as well as another to show whether the vote is persistent or not for convenience.

**Algorithm 13** Committee.Verify

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- vk: Voter's verifying key whose eligibility is being checked,
- $\pi$: Proof of eligibility.

**Output:**

- $b$: boolean set to 1 if the caller may vote, 0 otherwise.
- $w$: weight of the vote.
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

**Security.** When selecting committee members, there is an additional risk of under or over-representing their weight in the committee with respect to their actual stake in the network. An adversary can make use of this over-representation to increase their chances of controlling the committee, and in our case generating a certificate on an invalid EB. *Adaptive adversaries* are defined as adversaries that may corrupt a party at any moment, given that they do not increase their overall stake. For instance, an adversary with 40% stake controls at slot #1 SPO A and B – and can propose blocks in their stand at that time, and at slot #2, SPO B and C relinquishing control of the other two. If the committee selection is public, an adversary can make use of this information and potential over-representation to gain greater weight in that period. As such, to counter adaptive adversaries when there is risk of over-representation, elections must be made private.

## 3.4 Voting

We present here the generic algorithms for vote management. More precisely, in Leios, a vote is composed of a signature on the EB, the corresponding verifying key and optionally a proof of eligibility if the voter is a NPV. We use BLS signatures with the same key and domain separator for simplicity and compactness, as all votes can then be aggregated thanks to BLS signature batching.

**Vote generation.** To generate a vote, c.f. Algorithm 14, an SPO first checks their eligibility for the election by calling Committee.Eval. If they determine at the beginning of the committee period that they are a PV they may skip this check. If they are eligible, they generate a BLS signature on the EB and return the vote as the concatenation of the signature, their verifying key and the proof of eligibility if there is any.

**Algorithm 14** Vote.Sign

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- m: Message to sign, i.e. the EB's hash,
- $(\text{sk}^i_{\text{bls}},\ \text{vk}^i_{\text{bls}})$: The BLS signing keys for SPO $i$.

**Output:** v the vote created by SPO.

```
v ← ⊥
(b, π) ← Committee.Eval(OSD, n, n_1, PVs, eid, sk^i_bls)
if b then // Any Voter
    σ_bls ← BLS.Sign(m, sk^i_bls)
    v ← (σ_bls, vk^i_bls, π)
return v
```

**Vote verification.** To verify an SPO's vote, c.f. Algorithm 15, a party will first check if the corresponding voter was eligible to cast a vote for this election, and if their weight is non-null, through Algorithm 13. If so, they verify the BLS signature on the EB. A vote is considered valid only if both these checks are successful. The verification algorithm outputs a bit showing whether all checks have passed, the vote's weight and another bit to determine whether the voter was a PV or not for this report's convenience.

**Algorithm 15** Vote.Verify

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- m: Message to sign, i.e. the EB's hash,
- v: Vote to verify.

**Output:** $\top/\bot$ and weight $w$.

- $b$: boolean set to 1 if the voter may vote, 0 otherwise.
- $v$: vote's weight
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

1: $(\sigma_{bls}, \text{vk}^i_{\text{bls}}, \pi) \leftarrow \text{v}$
2: $(b_{com}, w, b_v) \leftarrow \text{Committee.}\mathsf{Verify}(\text{OSD}, n, n_1, \text{PVs}, eid, \text{vk}^i_{bls}, \pi)$
3: $b \leftarrow \mathsf{BLS.Verify}(\text{m}, \sigma_{\text{bls}}, \text{vk}^i_{\text{bls}})$
4: **return** $(b \wedge b_{com} \wedge w \neq 0, w, b_v)$

## 3.5 Certifying an EB

Once an SPO votes on an EB, they broadcast their vote so that it is verified and aggregated into a certificate by future slot leaders. The certificate is then broadcast to the whole network when attached to an RB.

**Remark.** To further limit the vote communication overhead, it may be possible to share the votes to even fewer committee members. Indeed, to prevent adversaries from blocking certificate generation, we only need to ensure that one honest SPO receives all votes, and may require more for robustness. We can furthermore reduce any margin of error thanks to the knowledge of which SPO is active, by monitoring the network and checking if any PV has voted. Setting the certificate threshold $\tau$ high enough would ensure on its own that the EB is known well in advance and so cutting the "certificate generation committee" would not impact any diffusion requirements.

**Certificate generation.** To generate a certificate, c.f. Algorithm 16, one needs to collect enough valid votes, that is votes representing more stake than the certificate threshold $\tau$ and aggregate the BLS signature on the EB. The verification keys and proofs of eligibility are appended to this multi-signature, the whole tuple corresponding to the certificate.

**Algorithm 16** Cert.Sign

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \mathrm{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- m: Message to vote / certify,
- $\tau$: Certificate threshold,
- $\{\mathrm{v}_i\}$, Unique votes' list (one vote per vk).

**Output:** $\mathrm{c}_{eid}$ certificate for $eid$, or $\perp$ if no certificate.

```
w ← 0; vec_PV ← []; vec_NPV ← []
for v_j ∈ {v_i} do
    (b_j, w_j, b_{j,v}) ← Vote.Verify(OSD, eid, n, n_1, PVs, eid, m, v_j)
    if b // the vote is valid then
        w = w + w_j
        if b_{j,v}, then vec_PV ← vec_PV ∪ v_j else vec_NPV ← vec_NPV ∪ v_j
if w ≥ τ then
    σ_bls ← BLS.BatchSig(vec_PV.vk‖vec_NPV.vk, vec_PV.σ‖vec_NPV.σ_bls)
    return c = (σ_bls, vec_PV.vk, vec_NPV.π, vec_NPV.vk)
return ⊥
```

**Remark.** Note that we can use binary arrays to represent the certificate's voters as all SPOs are supposed to store the ordered stake distribution.

**Certificate verification.** To verify a certificate, c.f. Algorithm 17, one needs to check the eligibility of each voter, that the certificate weight is larger than the threshold $\tau$, and that the BLS multi-signature on the EB verifies successfully.

**Algorithm 17** Cert.Verify

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- m: Message to vote / certify,
- $\tau$: Certificate threshold,
- c: Candidate certificate with unique vks.

**Output:** $b$: boolean set to 1 if the certificate is valid, 0 otherwise.

```
b ← ⊤; w ← 0;
(σ_bls, vec_PV.vk, vec_NPV.π, vec_NPV.vk) ← c
for vk^i_bls ∈ vec_PV do
    (b_i, w_i, _) ← Committee.Verify(OSD, n, n_1, PVs, eid, vk^i_bls, ⊥)
    b ← b ∧ b_i; w ← w + w_i
for j ∈ [|vec_NPV|] do
    σ_j ← vec_NPV[j].π; vk^j_bls ← vec_NPV[j].vk
    (b_j, w_j, _) ← Committee.Verify(OSD, n, n_1, PVs, eid, vk^j_bls, σ_j)
    b ← b ∧ b_j; w ← w + w_j
vk_c ← BLS.BatchKey(vec_PV.vk||vec_NPV.vk)
b_bls ← BLS.Verify(m, σ_bls, vk_c)
b_τ ← w ≥ τ
return b ∧ b_bls ∧ b_τ
```

# 4 Probabilistic voting with Fait accompli

We now turn to the use of Fait Accompli [5] for committee selection. This approach constitutes a modular protocol that can be integrated into existing systems while offering stronger security guarantees than simpler selection mechanisms. In particular, we are interested in the *weighted* variant, wFA, that is particularly efficient and greatly reduces the need of eligibility proofs.

## 4.1 Committee Size

As stated before, the initial committee size, around $\tilde{6}00$ SPOs, was computed from IID analysis to reach good probabilities (resp. negligible) on getting a honest (resp. adversarial) quorum. Since Fait Accompli offers less over-representation risks than IID, we could decrease, in principle, the committee size without sacrificing security. This, however, would lower the overall committee stake and so potentially decrease liveness, the chances of an adversary preventing the generation of a valid certificate.

More particularly, FA paper gives a loose bound $\delta$ on the probability of an adversary $\mathcal{A}$ being over-represented by a factor $\epsilon$, that is $\max_{\mathcal{A}} \Pr(\#_A(C) \geq$

$(\mathrm{s}_{\mathcal{A}} + \epsilon) \cdot \mathrm{s}_{tot}) \leq \delta = e^{-2n\epsilon^2}$ where $\#_{\mathcal{A}}(C)$ is the number of seats the adversary $\mathcal{A}$ owns in the committee $C$. Equivalently, for a security parameter $\lambda$ and over-representation $\epsilon$, we have that the committee size must be greater than $n \geq \frac{\lambda \cdot \ln 2}{2 \cdot \epsilon}$.

For instance, if we want a certificate to represent 75% of the total active stake, and consider an adversarial stake of 50%, the adversary needs to be over-represented by 25% to produce a certificate on an invalid EB, hence we set $\epsilon = 0.25$. Say we want the probability of this event to be lower than $\lambda = 2^{-128}$, this leads to a minimum committee size of 178 SPOs. Note that currently, 111 SPOs would not represent enough stake for generating a certificate, however.

## 4.2 Committee Selection

In a nutshell, given a stake distribution ordered non-increasingly per stake share, and a target committee size of $n$ seats, a $\mathsf{wFA^F}$ mechanism as defined in [6] defines a *cut-off* index $n_1 \leq n$. Every stakeholder with index $i < n_1$ in the ordered stake distribution gets a permanent vote. Every stakeholder with $n_1 \leq i < n$ is probabilistically assigned a seat in the committee as per a fallback mechanism $F$. Parties with an index $i < n_1$ are called Persistent Voters (PVs), and the remaining parties, with an index $n_1 \leq i \leq n$, the Non-Persistent Voters (NPVs). Intuitively, $n_1$ is computed by estimating the index after which switching to the fallback mechanism $F$ minimizes the adversary's advantage with respect to sticking to deterministic assignment.

**Fait Accompli Setup.** Let $\mathsf{Err_F}(\mathrm{OSD}, n)$ be the expected error (adversary advantage) when using algorithm $\mathsf{F}$ to decide a committee of size $n$. The setup of Fait Accompli, as shown in Algorithm 18, computes the cut-off $n_1$ by increasing the set of PVs until the expected error of Fait Accompli for picking the remaining committee on the unassigned SPOs exceeds the expected error when using the fallback on its own (without Fait Accompli). A concrete approximation on how to estimate this error is given in [6].

**Algorithm 18** $\mathsf{wFA^F.Setup}$, derived from [6] and [2].

**Input:**

- OSD: List of $(p_i, \mathrm{vk}^i_{\mathrm{bls}}, \mathrm{s}_i)$ pairs, mapping parties to their normalized stake, in non-increasing order per stake distribution.
- $n$: Target committee size

**Output:**

- $n_1$: the number of PVs,
- $[(p_1, w_1), \ \ldots, (p_{n_1}, w_{n_1})]$: The weighted list of persistent voters.

$n_1 \leftarrow \min_i \left(\mathsf{Err_{wFA}}(\mathrm{OSD}[i:], n-1) \geq \mathsf{Err_F}(\mathrm{OSD}[i:], n-1)\right)$
$\mathrm{PV} := [\perp; n_1]$ ; $\forall i \leq n_1, \ \mathrm{PV}[i] \leftarrow \mathrm{OSD}[i]$
**return** $(n_1, \mathrm{PV})$

**Fait Accompli evaluation and verification.** These functions depends on the fallback.

## 4.3 Fait Accompli with IID ($\mathsf{wFA^{IID}}$)

$\mathsf{wFA^{IID}}$ **Design.** We begin by considering the first weighted variant of Fait Accompli, which employs the IID protocol as a fallback mechanism. This construction is appealing due to its conceptual simplicity, and the public nature of the elections contributes to minimizing certificate size. However, despite these advantages, the scheme does not achieve adaptive security and all parties, PVs and NPVs, can be DoS'ed as they are known in advance.

$\mathsf{wFA^{IID}}$ **Evaluation.** We present in Algorithm 19 the evaluation algorithm that determines whether the caller's is eligible to vote for a given election *eid*.

**Algorithm 19** $\mathsf{wFA^{IID}.Eval}$

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \mathrm{PVs})$: Target committee size, and number and list of PVs,
- *eid*: Election ID,
- $\mathrm{sk}^i_{bls}$: Signing key of the caller $i$,

**Output:**

- $\top/\perp$: boolean set to 1 if the caller is eligible, 0 otherwise,
- $\pi$: proof of eligibility, may be $\perp$ if not needed.

**if** $i \leq n_1$, **return** $(1, \perp)$
weights $\leftarrow \mathsf{IID.Eval}(n - n_1, \mathrm{OSD}[n_1:].\mathrm{stake}, eid)$
**return** $(\mathrm{weights}[i - n_1] \neq 0, \perp)$

$\mathsf{wFA}^{\mathsf{IID}}$ **Verification.** We present in Algorithm 20 the corresponding verification algorithm that confirms whether a voter is legitimate or not for a given election $eid$.

**Algorithm 20** $\mathsf{wFA}^{\mathsf{IID}}.\mathsf{Verify}$

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{vk}^i_{bls}$: Verification key of the voter $i$,
- $\pi$: Proof of elibility

**Output:**

- $b$: boolean set to 1 if the caller may vote, 0 otherwise,
- $w$: weight of the vote,
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

$b \leftarrow \text{OSD}[i].\text{vk} \stackrel{?}{=} \text{vk}^i_{bls}$
**if** $i \leq n_1$ // the voter is a PV, **return** $(b, \text{OSD}[i].\text{stake}, 1)$
$\text{weights} \leftarrow \mathsf{IID}.\mathsf{Eval}(n - n_1, \text{OSD}[n_1 :], eid)$
**return** $(b \wedge \text{weights}[i - n_1] \neq 0, \text{weights}[i - n_1], 0)$

**Security** While the $\mathsf{wFA}^{\mathsf{IID}}$ construction achieves security, it falls short of stronger, adaptive guarantees typically desired in adversarial settings such as in Ouroboros Praos. In particular, its security relies on a static corruption model: the adversary must commit to the set of corrupted parties prior to the execution of the protocol. Under this assumption, the scheme ensures that the distribution of weights is respected, provided the underlying randomness beacon used in the IID sampling primitive is secure.

In summary, $\mathsf{wFA}^{\mathsf{IID}}$ offers better performances with the same security over simple binomial sampling (c.f. Section 4.1), at the cost a modest increase in complexity, but share important limitations agains adaptive adversaries.

## 4.4 Fait Accompli with Local Sortition ($\mathsf{wFA}^{\mathsf{LS}}$)

$\mathsf{wFA}^{\mathsf{LS}}$ **Design.** In this section, we now look at using local sortition $\mathsf{LS}$ as fallback $F$ of weighted Fait Accompli. Instead of doing a toss coin on each of the $n - n_1$ NPV seats (with probability the user's relative stake over the total stake minus the persistent stake), local sortition outputs directly the number of seats a user has won based on their lottery ticket, modeled by a VRF and a binomial distribution $\mathsf{B}$. More particularly, each user plays Algorithm 7 with $n = n - n_1$ and s the normalized non-persistent stake to determine its eligibility, and vote verifiers run Algorithm 8 to verify these eligibility proofs.

$\mathsf{wFA}^{\mathsf{LS}}$ **Evaluation.** We present in Algorithm 21 how to determine voting eligibility for $\mathsf{wFA}^{\mathsf{LS}}$. Either the voter has enough stake to be part of the PVs, or they run locally sortition evalution and may create a vote if and only if their weight in the committee is non-null.

**Algorithm 21** $\mathsf{wFA}^{\mathsf{LS}}.\mathsf{Eval}$

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{sk}^i_{bls}$: Signing key of the caller,

**Output:**

- $\top/\bot$: boolean set to 1 if the caller is eligible, 0 otherwise,
- $\pi$: proof of eligibility, may be $\bot$ if not needed.

**if** $i \leq n_1$, **return** $(1, \bot)$
$(w, \pi) \leftarrow \mathsf{LS.Eval}(n - n_1, eid, \text{OSD}[i].\text{s}, \text{sk}^i_{bls})$
**return** $(w \neq 0, \pi)$

$\mathsf{wFA}^{\mathsf{LS}}$ **Verification.** Similarly, we present in Algorithm 22 how to check whether a voter was indeed eligible for voting using $\mathsf{wFA}^{\mathsf{LS}}$. Either the voter has enough stake to be part of the PVs, or they run locally sortition verification and check if their weight in the committee is non-null.

**Algorithm 22** $\mathsf{wFA}^{\mathsf{LS}}.\mathsf{Verify}$

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{vk}^i_{bls}$: Signing key of the caller,
- $\pi$: Proof of elibility

**Output:**

- $b$: boolean set to 1 if the caller may vote, 0 otherwise,
- $w$: weight of the vote,
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

$b \leftarrow \text{OSD}[i].\text{vk} \stackrel{?}{=} \text{vk}^i_{bls}$
**if** $i \leq n_1$ // the voter is a PV, **return** $(b, \text{OSD}[i].\text{stake}, 1)$
$(w, b_{\mathsf{LS}}) \leftarrow \mathsf{LS.Verify}(n - n_1, eid, \text{OSD}[i].\text{stake}, \text{vk}^i_{bls}, \pi)$
**return** $(b \wedge b_{\mathsf{LS}}, w, 0)$

**Security.** The $\mathsf{wFA}^{\mathsf{LS}}$ variant leverages local sortition to determine eligibility privately, which enhances unpredictability compared to the $\mathsf{IID}$-based approach. Since selection is computed locally and only revealed upon participation, an adversary cannot know the whole committee in advance, limiting pre-selection attacks. Furthermore, by rotating signing keys after each election, the scheme ensures that any compromise of a party's secret keys affects only the current round and cannot influence future selections. These two technique combined, private elections and rotating keys, make $\mathsf{wFA}^{\mathsf{LS}}$ *adaptively secure*. In practice, this means that even an adversary capable of dynamically corrupting parties during the protocol execution cannot meaningfully bias the committee beyond a negligible probability over time.

Without rotating the $\mathsf{BLS}$ keys at each election, $\mathsf{wFA}^{\mathsf{LS}}$ is no longer adaptive secure. Indeed, even after an adversary ceases corrupting an honest party, they still retain access to that party's keys. This allows them to determine whether the party is eligible to vote and refine their corruption strategy accordingly, and forge votes on the party's behalf. Thus, while security may hold for a single round, the scheme fails to provide security across multiple rounds.

In practice, a relaxed version of adaptive security may be sufficient, as in Cardano, SPOs are required to update their $\mathsf{KES}$ keys only every 36 hours, and change them every 90 days. In the current prototype, the $\mathsf{BLS}$ keys are registered alongside the $\mathsf{KES}$ ones, which makes it straightforward to implement a relaxed $\mathsf{BLS}$ key rotation (registering fresh $\mathsf{BLS}$ keys every 90 days) without introducing further complexity or overhead.

In summary, $\mathsf{wFA}^{\mathsf{LS}}$ is secure against adaptive security at the cost of introducing eligibility proofs for NPVs in both their votes and the resulting certificate. The protocol is also more complex, because of the dissociation of voters between PVs and NPVs. Lastly, we made the arbitrary choice, for simplicity and ease of reading, of using the same keys for both determining and proving elibigibility, and voting on an EB (by signing it). This goes contrary with Cardano's current protocol which defines distinct keys for the slot leader election, the $\mathsf{VRF}$ keys, and for block production, the $\mathsf{KES}$ keys. Were we to follow in that example, Leios becomes even more complex, unless we merge both $\mathsf{VRF}$ and vote eligibilty keys together.

# 5 Unrestricted Voting

Backtracking to simpler strategies for committee selection, we now look at full, unrestricted voting, that is the second option in [4] where *all* SPOs vote. We could argue that there is no committee selection, or it is the identity.

This means that vote generation and verification are identical for all participants, since there is no distinction between NPVs and PVs. Furthermore, a vote's weight is directly determined by the stake the voter holds and as such there is no over-representation and adaptive security may be achieved with key rotation alone.

All **Setup.** The setup, as shown in Algorithm 23, is nearly non-existent as we simply set $n_1$ to the number of SPOs.

**Algorithm 23** All.Setup

**Input:**

- OSD: List of $(p_i, \mathrm{vk}^i_{\mathrm{bls}}, \mathrm{s}_i)$ pairs, mapping parties to their normalized stake, in non-increasing order per stake distribution.
- $n$: Target committee size

**Output:**

- $n_1$: the number of PVs,
- $[(p_1, w_1), \ \ldots, (p_{n_1}, w_{n_1})]$: The weighted list of persistent voters.

1: **if** $n \neq \|\mathrm{OSD}\|$, **return** $(-1, \perp)$
2: **return** $(n, \mathrm{OSD})$

All **Evaluation.** The evaluation, as shown in Algorithm 24, is also straightforward, as the weight is directly defined as the voter's stake.

**Algorithm 24** All.Eval

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \mathrm{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\mathrm{sk}^i_{bls}$: Signing key of the caller,

**Output:**

- $\top/\perp$: boolean set to 1 if the caller is eligible, 0 otherwise,
- $\pi$: proof of eligibility, may be $\perp$ if not needed.

**return** $(\mathrm{OSD}[i].\mathrm{stake} \neq 0, \perp)$

All **verification.** The verification, as shown in Algorithm 25, is the same as the evaluation, we only need to check that the party is indeed registered and their stake is non-null.

**Algorithm 25** $\mathsf{All.Verify}$

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{vk}^i_{bls}$: Signing key of the caller,
- $\pi$: Proof of eligibility

**Output:**

- $b$: boolean set to 1 if the caller may vote, 0 otherwise,
- $w$: weight of the vote,
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

$b \leftarrow \text{OSD}[i].\text{vk} \stackrel{?}{=} \text{vk}^i_{bls}$
$w_i \leftarrow \text{OSD}[i].\text{stake}$
**return** $(b \wedge w_i \neq 0, w_i, 1)$

**Security.** Since voter weights are derived directly from the stake distribution, there is no risk of over-representation. This limits the capacity of the adversary in forging a certificate to controlling the keys of enough participants – or breaking the signature scheme. This is at the cost of the worst communication overhead, as everyone vote.

To achieve security against adaptive security, it suffices to to rotate keys.

# 6 Public Persistent Committee Selection

To reduce the voting communication cost, we finally look back at *publicly selecting* a committee composed of *persistent* voters only.

We could select these directly with $\mathsf{IID}$ but a simpler method consists in choosing the voters until we reach a certain cumulative stake. The most efficient way, in order to have as few voters as possible, is in ordering them in non-increasing order of stake and pick them one by one. We could, however, for fairness, randomize the ordering at the cost of more votes being broadcasted.

We can liken this protocol to $\mathsf{wFA}$, where only PVs are allowed to vote and SPOs that do not make the cut are directly ignored, without any fallback mechanism. This comparison would, however, stop there, as the absence of fallback prevents the use of $\mathsf{wFA}$ "cut-off" mechanism and security analysis. We refer to this variant as "Truncation," and analyze it next.

## 6.1 Using $\mathsf{IID}$ only

Here, we consider the option of using independent and identically distributed ($\mathsf{IID}$) sampling to select persistent voters.

OnlyIID **Setup.** As shown in Algorithm 26, we take as input the target committee size and output the parties with non null weight.

**Algorithm 26** OnlyIID.Setup

**Input:**

- OSD: List of $(p_i, \mathrm{vk}^i_{\mathrm{bls}}, \mathrm{s}_i)$ pairs, mapping parties to their normalized stake, in non-increasing order per stake distribution.
- $n$: Target committee size

**Output:**

- $n_1$: the number of PVs,
- $[(p_1, w_1), \ \ldots, (p_{n_1}, w_{n_1})]$: The weighted list of persistent voters.

1: weights $\leftarrow$ IID.Eval$(n, \mathrm{OSD}, eid)$
2: PVs $\leftarrow$ [];
3: $\forall (v, w_i) \in \mathrm{OSD} \times \mathrm{weights}$, **if** $w_i \neq 0$, PVs $\leftarrow$ PVs $\cup (v, w_i)$
4: **return** $(\|\mathrm{PVs}\|, \mathrm{PVs})$

OnlyIID **Evaluation.** Contrary to usual, as shown in Algorithm 27, we only need the list of PVs for evaluating vote eligibility.

**Algorithm 27** OnlyIID.Eval

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \mathrm{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\mathrm{sk}^i_{bls}$: Signing key of the caller,

**Output:**

- $\top/\bot$: boolean set to 1 if the caller is eligible, 0 otherwise,
- $\pi$: proof of eligibility, may be $\bot$ if not needed.

**if** $\exists j$ *s.t.* $g_2^{\mathrm{sk}^i_{bls}} == \mathrm{PVs}[j].\mathrm{vk}$, **return** $(\mathrm{PVs}[j].\mathrm{weight} \neq 0, \bot)$
**return** $(0, \bot)$

OnlyIID **Verification.** Similarly to the evaluation, as shown in Algorithm 28, we only need the list of PVs to check whether a voter is eligible and retrieve their weight.

**Algorithm 28** OnlyIID.Verify

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{vk}^i_{bls}$: Signing key of the caller,
- $\pi$: Proof of eligibility

**Output:**

- $b$: boolean set to 1 if the caller may vote, 0 otherwise,
- $w$: weight of the vote,
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

$b \leftarrow \text{OSD}[i].\text{vk} \stackrel{?}{=} \text{vk}^i_{bls}$
**if** $\exists j$ *s.t.* $\text{vk}^i_{bls} == \text{PVs}[j].\text{vk}$ **then**
  **return** $(b \wedge \text{PVs}[j].\text{weight} \neq 0, \text{PVs}[j].\text{weight}, 1)$
**return** $(0, 0, 1)$

**Security.** This approach is not adaptive secure as there is a risk of over-representation and the weights are publicly known, even if the keys are rotated. As the committee is public and reduced in size, the communication cost is reduced but at a greater risk of liveness, absentees and DoS attacks being more impactful here than when everyone votes. The protocol remains relatively simple and the voting communication overhead is lower than for unrestricted voting.

## 6.2 Truncation

We parameterize this protocol by a minimum committee stake, $\theta$, and select stake pool operators (SPOs) as voters by ordering them in non-increasing stake and including them until their cumulative stake reaches $\theta$ percent.

The advantage of this approach, compared to imposing a cut-off on individual stake, is that it avoids the need to readjust parameters as the stake distribution evolves.

Truncation **Setup.** *Contrary to the usual setup*, as shown in Algorithm 29, we take as input the target committee stake $\theta$ instead of the target committee size.

**Algorithm 29** Truncation.Setup

**Input:**

- OSD: Ordered stake distribution,
- $\theta$: Target committee stake

**Output:**

- $n_1$: the number of PVs,
- $[(p_1, w_1), \ \ldots, (p_{n_1}, w_{n_1})]$: The weighted list of persistent voters.

```
stake ← 0; n_1 ← 0
for (p_i, vk^i_bls, s_i) ∈ OSD do
    if stake ≤ θ then
        stake ← stake + s_i
        n_1 ← n_1 + 1
        else break
return (n_1, OSD[: n_1])
```

Truncation **Evaluation.** The evaluation, as shown in Algorithm 30, consists in checking if the caller's is among the $n_1$ richest SPOs.

**Algorithm 30** Truncation.Eval

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{sk}^i_{bls}$: Signing key of the caller,

**Output:**

- $\top/\bot$: boolean set to 1 if the caller is eligible, 0 otherwise,
- $\pi$: proof of eligibility, may be $\bot$ if not needed.

```
if i ≥ n_1, return (⊥, ⊥)
return (OSD[i].stake ≠ 0, ⊥)
```

Truncation **Verification.** The verification, as shown in Algorithm 31, consists in checking whether the voter is registered and among the $n_1$ richest SPOs. The vote weight is set as the SPO's stake and the voter is considered persistent.

**Algorithm 31** Truncation.Verify

**Input:**

- OSD: Ordered stake distribution,
- $(n, n_1, \text{PVs})$: Target committee size, and number and list of PVs,
- $eid$: Election ID,
- $\text{vk}^i_{bls}$: Signing key of the caller,
- $\pi$: Proof of eligibility

**Output:**

- $b$: boolean set to 1 if the caller may vote, 0 otherwise,
- $w$: weight of the vote,
- $b_v$: boolean set to 1 if the voter is persistent, 0 otherwise.

$b \leftarrow \text{OSD}[i].\text{vk} \stackrel{?}{=} \text{vk}^i_{bls} \wedge i < n_1$
$w_i \leftarrow \text{OSD}[i].\text{stake}$
**return** $(b \wedge w_i \neq 0, w_i, 1)$

**Security.** Contrary to the previous case, as there is no risk of over-representation, this approach can be adaptive secure through key rotation. Both liveness and fairness are negatively impacted.

The protocol is, however, simpler and the voting communication overhead may be comparable, if the target committee size is the similar.

# 7 Conclusion

Here, we present our key findings from formalizing and analyzing all committee variants, along with recommendations for selecting among them and additional remarks.

## 7.1 Key findings

While the $\mathsf{wFA}^{\mathsf{LS}}$ algorithm offers the highest security, potentially supporting adaptive security, it is, in practice, the least efficient in terms of communication overhead, certificate size, and verification cost. Tighter security bounds for wFA may be reached by using fallback specific security bounds rather than the loose generic approximation when defining the cut-off. This, however, is arguably less critical, as the diffusion requirements already necessitate higher values for the certificate's stake threshold $\tau$. IOG's research team is working on a follow-up paper to Fait Accompli that, instead of adjusting the cut-off over risk of over-representation for security, focuses directly on committee size for a fixed acceptable error.

The remaining probabilistic options – $\mathsf{wFA}^{\mathsf{IID}}$, OnlyIID and Truncation– are secure under the static adversary model as long as the committee size, or cor-

responding stake, is large enough. As for All and Truncation, they can achieve adaptive security at the cost of fairness. Among them, $\mathsf{wFA}^{\mathsf{IID}}$ is the most complex, All the most expensive and both OnlyIID and Truncation appear to be a compromise between them with OnlyIID being more slightly complex but fairer.

In Table 7.1, we loosely compare the proposals over security, liveness, fairness, code complexity, vote efficiency – both the number and size of the votes as well as the certificate size. In particular, $\mathsf{wFA}^{\mathsf{LS}}$ may have a comparable number of votes than most proposals but their NPVs votes are larger. The code complexity of $\mathsf{wFA}^{\mathsf{LS}}$ is the highest as the fallback is run at every election, $\mathsf{wFA}^{\mathsf{IID}}$ is slightly more complex than OnlyIID because of its hybrid nature but also more secure and efficient.

| | $\mathsf{wFA}^{\mathsf{LS}}$ | $\mathsf{wFA}^{\mathsf{IID}}$ | OnlyIID | All | Truncation |
|---|---|---|---|---|---|
| Security | Adaptive | Static | Static | Adaptive | Adaptive |
| Liveness | Medium | Low | Low | High | Low |
| Fairness | Medium | Medium | Medium | High | Low |
| Complexity | High | Medium | Medium | Low | Low |
| Vote Efficiency | Medium | High | High | Low | High |
| Certificate size | High | Low | Low | Low | Low |

Table 1: Committee selection protocol comparisons.

Finally, to achieve adaptive security, regardless of the chosen protocol, it is *essential* however to ensure that BLS keys are updated regularly to achieve adaptive security. Since KES keys are not rotated at every call, it may suffice to rotate BLS keys less frequently, resulting in a relaxed form of adaptive security.

## 7.2 Recommendations

**BLS key revocation.** So far, BLS keys have been assumed to be registered via a KES signature. Consequently, if KES keys are revoked (e.g., by preemptively uploading a new operational certificate), the corresponding BLS keys should also be revoked. We omit the details of BLS key revocation for now, as they are not defined in the Leios paper or specifications. We may additionally push this logic further to encapsulate key rotation so that BLS keys are always registered with the up-to-date KES keys.

**Key scope in $\mathsf{wFA}^{\mathsf{LS}}$.** In the $\mathsf{wFA}^{\mathsf{LS}}$ proposal, BLS keys are used for both eligibility check and the signing of the endorsment blocks. As the eligibility check is analogous to the slot leader elections, the existing VRF key could be used for this purpose, reserving BLS keys solely for signing endorsement blocks. This approach improves separation of duties by using distinct keys for distinct operations. Alternatively, domain separation tags could be employed if maintaining a Leios-specific key is preferred.

### 7.3 Further observations

**Certificate generation.** At the cost of making Leios more complex and reducing liveness, we may choose to select few vote aggregators to limit the vote communication overhead (multicasting vote and broadcasting certificates).

**VRF rotation.** VRF keys, used for slot leader selection – independently of Leios, are currently not rotated and cannot neither be revoked nor updated. While this does not decrease the security of publishing blocks on the main chain, it facilitates grinding attacks. We would recommend to register fresh VRF keys every time a new operational certificate is published with new KES keys. This would also have the merit of unifying key management in Cardano.

**Adaptive security.** Ouroboros Praos' Common Prefix (CP) property consists in guaranteeing that the state of the blockchain $k$ blocks ago is consistent across parties. This implies that blocks older than these are considered valid, and the probability of a roll back past $k$ blocks is negligible. As Leios acts as an extension of Praos for publishing transactions on-chain, the protocol should also guaranty the Common Prefix property. This implies that adaptive security should be necessary to prevent the adversary from generating rogue certificates by silently collecting keys.

More research may be needed to check how critical adaptive security is for Leios by reviewing both Ouroboros and Ouroboros Praos papers. Among the current proposals, only $\mathsf{wFA}^{\mathsf{LS}}$, All and Truncation with BLS key rotation would provide adaptive security, should it be needed.

## References


[1] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In Colin Boyd, editor, *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings*, Lecture Notes in Computer Science, pages 514–532. Springer, 2001. doi:10.1007/3-540-45682-1_30.

[2] Brian Bush. Ouroboros leios voting crypto benchmarks, 2025. URL: https://github.com/input-output-hk/ouroboros-leios/blob/d5f1a9bc940e69f406c3e25c0d7d9aa58cf701f8/crypto-benchmarks.rs.

[3] Brian Bush. Ouroboros leios voting crypto specification, 2025. URL: https://github.com/input-output-hk/ouroboros-leios/blob/d5f1a9bc940e69f406c3e25c0d7d9aa58cf701f8/crypto-benchmarks.rs/Specification.md.

[4] Paul Clark. Leios voting committee selection options, 2026. URL: https://docs.google.com/document/d/1CmSC8BRDZTH32fk_bZ4db6baYGodZMB-71b-S18B2PQ.

[5] Peter Gaži, Aggelos Kiayias, and Alexander Russell. Fait accompli committee selection: Improving the size-security tradeoff of stake-based committees. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 845–858, 2023.

[6] Peter Gazi, Aggelos Kiayias, and Alexander Russell. Fait accompli committee selection: Improving the size-security tradeoff of stake-based committees. *IACR Cryptol. ePrint Arch.*, 2023:1273, 2023. URL: `https://eprint.iacr.org/2023/1273`.

[7] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th symposium on operating systems principles*, pages 51–68, 2017.

[8] Tal Malkin, Daniele Micciancio, and Sara Miner. Composition and efficiency tradeoffs for forward-secure digital signatures. *Cryptology ePrint Archive*, 2001.